

Project Proposal

AgentDesign: Hermes Profile Architecture

Math & Librarian Profiles — Design Documenta

Mehdi Ghasemi

Project Proposal

September 2026

Executive Summary

This document records the complete design of the two Hermes agent profiles that power my mathematical research workflow — `math` and `librarian` — and specifies an abstraction layer under which either profile can be re-pointed at any future math project without modification. All facts are extracted from the live configuration on `/home/mehdi/.hermes/profiles/` (config version 49, September 2026); every claim in Sections ??–?? is verifiable against the files listed in Section ??.

The architecture resolves into five layers: a shared *Hermes core*, an isolated *profile system*, a *skill catalog* (57–59 skills, capability-gated per profile), an *MCP bridge* (13 registered servers, one generic CLI-adaptor pattern covering 8 of them), and an *external service plane* on the local network. The math profile is a full-stack researcher with a 5.9 GB sandboxed toolchain environment; the librarian profile is a capability-degraded derivative that shares every registration but disables all computational skills and adds two ingestion skills of its own.

Two structural findings drive the abstraction design: (i) the librarian profile’s MCP servers import scripts from the math profile’s directory tree, so it cannot exist without the math profile; and (ii) both profiles contain a stale reference to `IreneRewrite`, deleted when that worktree was merged into `Irene` on 2026-09-18. Section ?? proposes a capability/manifest model — persona templates, path interpolation, a shared adaptor, a service registry, and per-project binding files — that removes every identified coupling in five concrete steps.

Contents

1 Objectives and Scope

The AgentDesign project has three objectives:

1. **Document the as-is architecture.** Record, with file-level evidence, how the math and librarian profiles are composed: directory layout, configuration sections, skill catalogs, MCP server registrations, external services, scheduled jobs, and state stores. This is Sections ??–??.
2. **Identify the coupling points.** Enumerate every place where a profile depends on the specific current project layout — absolute paths, cross-profile imports, hardcoded endpoints, and stale references — so that the design of record distinguishes what is load-bearing from what is incidental (Section ??).
3. **Specify an abstraction layer.** Define how profiles become reusable personas: independent of any particular math project, instantiable for a new project with zero or one line of configuration, and testable against a capability contract (Section ??).

The scope is the two active research profiles. The remaining four registered profiles (default, coder, advisor, trader) are out of scope; their existence is noted only in Section ??. This project does not modify any live profile configuration — it is a documentation and design artifact.

2 Layered Architecture

Figure ?? gives the visual layout of the infrastructure under which both profiles operate. Five layers are distinguished; each lower layer is shared, each upper layer is isolated per profile at `~/.hermes/profiles/<name>/`.

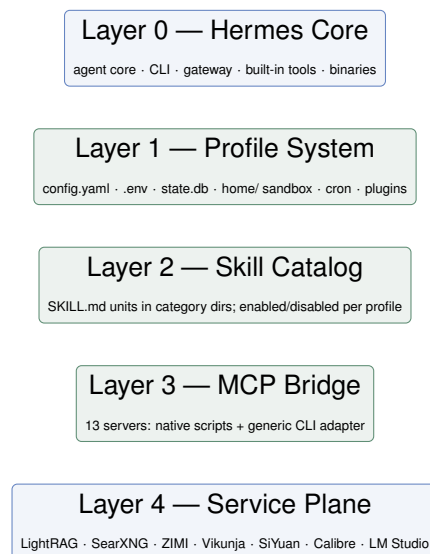


Figure 1: Five-layer profile architecture. Layers 2–3 are the capability surface of a profile; layers 0 and 4 are shared infrastructure. Arrows indicate the direction in which each layer is configured by, or depends on, the layer above it.

Layer 0 — Hermes Core

The core lives at `~/.hermes/hermes-agent/`: agent runtime, CLI (profile switching, tool registry), platform gateway (Telegram/WhatsApp/webhook adapters), built-in tools (terminal, browser, vision, file I/O, web

search, kanban, memory) and the MCP discovery layer (`tools/mcp_tool_*.py`). Shared binaries (tirth security scanner, `uv`, Python 3.14, Node 26, ripgrep, Chromium) live in `~/.hermes/tools/`; all MCP stdio servers run from a single shared virtualenv at `~/.hermes/mcp-servers/.venv/`.

Layer 1 — Profile System

Each profile is an isolated directory with its own configuration, credentials, state and sandbox. The two profiles under study:

- **math** (8.4 GB; 493 sessions): full-stack mathematical researcher. Working directory `/home/mehdi/Code/Python/`. Carries a 5.9 GB sandboxed home environment with the complete local toolchain (Section ??). Active Telegram gateway; five cron jobs registered (two enabled). Kanban orchestrator and default assignee is `math`.
- **librarian** (174 MB; 8 sessions): literature manager — “manage and maintain books and articles.” Working directory `/home/mehdi/Code/`. Empty `home/` stub, no cron jobs, no gateway platforms. Shares the same kanban assignment (`math`), which is itself a coupling noted in Section ?? . It is also the only profile with an active metrics sender: its `telemetry.shared_metrics` block has both `enabled` and `send` set to `true`; `math`’s config carries no telemetry block at all.

Both profiles run the same local model: `qwen3.8-27b` via LM Studio at `http://localhost:1234/v1`, with an `on_session_start` hook (`~/.hermes/agent-hooks/ensure-lmstudio.sh`) that guarantees the model server is up before each session. Long-term memory in both profiles is backed by Honcho (`memory.provider: honcho`, service at port 8008).

Layer 2 — Skill Catalog

Skills are Markdown-defined capability units (`SKILL.md` with YAML frontmatter), organized in category directories, optionally carrying scripts and credential files. Each profile holds 57 skill units across nine named categories plus a handful of top-level skills (union: 59); the full per-profile inventory is in `references/skill_inventory.md`; Section ?? summarizes the design-relevant parts.

Layer 3 — MCP Bridge

Skills reach services either directly (terminal/shell) or through Model Context Protocol servers registered under `mcp_servers`: in each profile’s config; every registered tool is exposed to the model as `mcp_<server><tool>` .Section ?? documents the two server